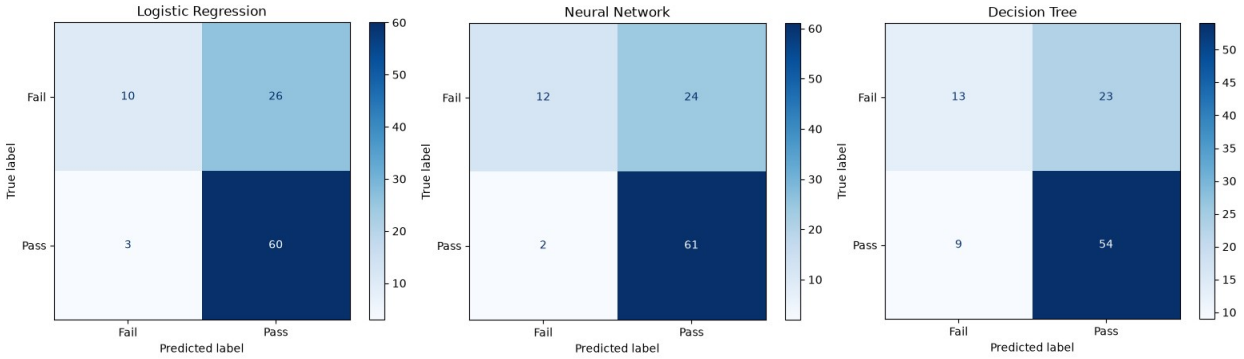
Task 1:

This I the accuary
Logistic Regression: 0.7070707070707071
Decision Tree: 0.6767676767676768
Neural Network: 0.7373737373737373



The three models had fairly similar accuracy. Logistic Regression achieved an accuracy of 0.707, the Decision Tree achieved 0.677, and the Neural Network achieved 0.737. This means that the Neural Network performed best on this particular test set. However, the difference between each model is only around 3 percentage points, so the improvement is relatively small. The difference between the best and worst model is about 6 percentage points.

Looking at the confusion matrices gives more information about how the models performed. The Neural Network correctly predicted 61 students who passed and only incorrectly predicted 2 passing students as failing. Logistic Regression correctly predicted 60 passing students and incorrectly classified 3 of them. The Decision Tree performed worse for this group, correctly predicting 54 and incorrectly predicting 9.

However, all three models had more difficulty predicting students who actually failed. Possibly because the dataset includes more examples of passing students. The Neural Network, for example, correctly predicted 12 failing students but incorrectly classified 24 of them as passing. This shows that the accuracy score does not tell the whole story.

The Neural Network is the most complex of the three models and did achieve the highest accuracy, but only by a small margin. Therefore, these results do not necessarily mean that a Neural Network is generally better. The result is based on one particular train/test split, and performance could be different with another split or dataset.

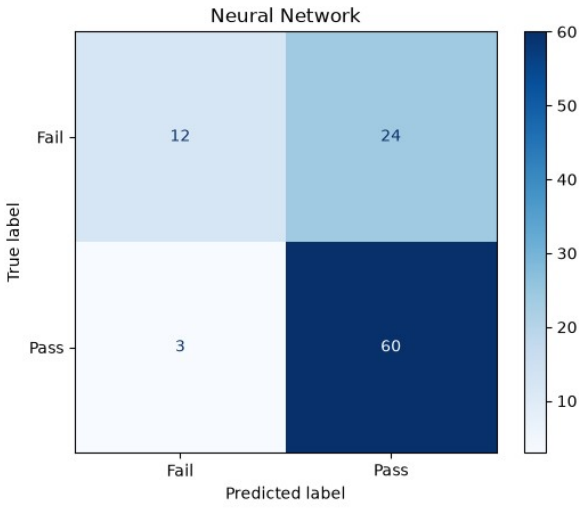
Task 2

For this experiment, I changed the neural network architecture from `(8,)` to `(8, 4)`. The original network had one hidden layer with 8 neurons, while the new network has two hidden layers, with 8 neurons in the first layer and 4 in the second. Everything else, including the predictors and train/test split, was kept the same.

The original neural network had an accuracy of 73.7%, while the new architecture achieved approximately 72.7%. This is a decrease of about one percentage point. The difference is therefore quite small, but adding another hidden layer did not improve the model in this experiment.

One possible explanation is that the additional complexity was not useful for this dataset. The model only uses three features: study time, previous failures, and absences. The relationship between these features and whether a student passes may not be complex enough to benefit from an additional hidden layer. A more complex network can learn more complicated patterns, but that does not necessarily mean that it will make better predictions.

This result does not clearly prove that the `(8,)` architecture is generally better. The difference is very small and the result is based on one particular dataset and train/test split. A different split or dataset could produce different results.



Task 3

Local search is usefull for finding a good soulution in a large searchspace. It works by starting with one soulution and working from that to a better one. This way of searching is called hill climbing. Take the travelling salesman problem forexample, if exchanging two cities gives a better result, then we “climb” the hill towards a better soulution and continue the search in that direction instead of a swap that would make the evaluation worse. One problem with this approach is that it can reach a local optimum. Say the hill climbing algorithm works and we reach the “top” of the hill. All next states are worse than what we already have found, can we then conclude that we have found the optimal soulution? The answer is

no, because we have only found a local optimum. The reason is that there could be a better soultion that requires us to travel down the hill, visit worse states, to reach a new peak. One way to solve this is with random restart. Random restart is where you start some dirfferent places in the search space so that you can find more hills, which gives you a better chance of finding the global optimum, or atleast something closer to it.

Task 4

Supervised machine learing learns patterns through the data and uses that to predict a result. In this case we will use study time, previous faliures and attendance, and the target variable is if the student passes or fails. The dataset should be divided into training and test data because the model needs one part of the data to learn from and another separate part to evaluate how well it performs. The model learns patterns from the training data, but if we tested it on the same data, the result could be misleading because the model has already seen those examples. The test data contains examples the model has not seen during training, so it gives a better indication of how well the model can generalize to new cases.

One way to test if the model is good is a confusion matrix. Using only percentages can give a false representaiton on how good the model is. The model can get a really high percentage if its good at predicting success and there are more examples of that case in the test data. The confusion matrix helps distinguish false positives and false negatives from correct predictions, and makes it easier to see how accurate the model is at predicting the different results.

If the model was set to predict the numerical score instead of pass/fail, this would be a regression problem rather than a classification problem because the model would need to predict a specific numerical value instead of a class such as 0 or 1. The training process would still use examples with known answers, but the evaluation would be different. Instead of accuracy and a confusion matrix, we could measure how far the predicted scores are from the actual scores, for example using mean absolute error.

Task 5

The input layer is the layer where the data gets entered, for the student example the input layer would countain 3 neurons, one for each feature. The hidden layer is where the proceesing happens. The hidden layer has a width and a height. Each neuron in the hidden layer recieves input from all neurons in the previous layer, that neuron multiplies each neuron by its weight, sums the results together, then adds the bias. The result is then passed through an activation function like ReLU or sigmoid. Then the output of that neuron becomes a part of the input of the neurons in the next layer. The weights of the neurons decide how important the input from a previous layer is. The bias is a value that is used to finetune the value before its sent to the activation function. Its important that the activation function is nonlinear for the network to learn complex patterns. Without a nonlear function the layers would just collapse into one linear transformation. The output layer is neurons that give a result. With the student example you could have just one neuron and if its result is > 0.5 its a pass and fail otherwise or for the neumerical

output to have one neuron that produces a numerical value. The foreward pass is just sending input through the network and getting an answer. The answer gets run through the loss function, then with backpropagation the network calculates how much each weight contributet to the error. Gradient decent uess that information to adjust the weights in a direction that would reduce the loss. This loop is repeated during training to reduce overall error. A neural network can overfit if it learns the training data too closely instead of learning patterns that generalize to new data. This can happen especially if the network is very complex compared with the amount of training data. One way to reduce overfitting is early stopping, where training is stopped when performance on validation data stops improving. A neural network is therefore not automatically better than a simpler model. A more complex network may take longer to train and can overfit, while a simpler model such as logistic regression may perform just as well on a relatively simple problem.